



NoSQL Database

Module 7

BCSE302L – Database Systems



Introduction to NoSQL

- NoSQL is a type of database management system (DBMS) that is designed to handle and store large volumes of unstructured and semi-structured data.
- NoSQL databases use flexible data models that can adapt to changes in data structures and are capable of scaling horizontally to handle growing amounts of data.



Introduction to NoSQL

- The term NoSQL originally referred to “non-SQL” or “non-relational” databases, but the term has since evolved to mean “not only SQL,” as NoSQL databases have expanded to include a wide range of different database architectures and data models.



Introduction to NoSQL

- NoSQL is an umbrella term for all DBs and Data stores that **don't follow the RDBMS principles**.
 - A class of products
 - A collection of several concepts about data storage and manipulation
 - Often related to large data sets



Where does NoSQL come from?

- Non-relational DBMD, a new incarnation represented by NoSQL
 - Due to massively scalable Internet applications
 - Based on distributed and parallel computing
- Development
 - Start with Google
 - First research paper published in 2003
 - Continuation by Lucene's developers/Apache (Hadoop) and Amazon (Dynamo)
 - More products from Facebook, Netflix, Yahoo, eBay, Hulu, IBM and many more.



Where does NoSQL come from?

- Dynamo and Big Table
- Three major papers were the seeds for NoSQL
 - Big Table (Google)
 - Dynamo (Amazon)
 - Distributed key-value data store
 - Eventual consistency
 - CAP Theorem



NoSQL and Big Data

- NoSQL comes from Internet, thus it is often related to the “big data” concept
- How much big are “big data”?
 - Over few terabytes Enough to start spanning multiple storage units
- Challenges
 - Efficiently storing and accessing large amounts of data is difficult, even more considering fault tolerance and backups
 - Manipulating large data sets involves running immensely parallel processes
 - Managing continuously evolving schema and metadata for semi-structured and un-structured data is difficult



Why are RDMS not suitable for Big Data

- The context is Internet
- RDBMSs assume that data are
 - Dense
 - Largely uniform (structured data)
- Data coming from Internet are
 - Massive and sparse
 - Semi-structured or unstructured
- With massive sparse data sets, the typical storage mechanisms and access methods get stretched

////////////////////////////////////

NoSQL Characteristics

- Large data volumes
 - Google's "big data"
- Scalable replication and distribution
 - Potentially thousands of machines
 - Potentially distributed around the world
- Queries need to return answers quickly
- Mostly query, few updates
- Asynchronous Inserts & Updates
- Schema-less
- ACID transaction properties are not needed – BASE
- CAP Theorem
- Open source development



NoSQL Database

- NoSQL originally referring to **non SQL or non relational** is a database that provides a mechanism for storage and retrieval of data.
- This data is **modeled** in means **other than the tabular relations used in relational databases**.
- Such databases **came into existence in** the late **1960s**, but did not obtain the NoSQL moniker until a surge of popularity in the early twenty-first century.



NoSQL Database

- NoSQL databases **are used in real-time web applications and big data** and their use are increasing over time.
 - NoSQL systems are also **sometimes called Not only SQL** to emphasize the fact that *they may support SQL-like query languages*.
 - A NoSQL database **includes simplicity of design, simpler horizontal scaling** to clusters of machines and **finer control over availability**.



NoSQL Database

- The **data structures used** by NoSQL databases **are different** from those used by default in relational databases **which makes some operations faster** in NoSQL.
- The suitability of a given NoSQL database depends on the problem it should solve.



NoSQL Database

- **Data structures used** by NoSQL databases are sometimes also viewed as more **flexible than relational database tables**.
- Many NoSQL stores **compromise consistency in favor of availability, speed and partition tolerance**.
- Barriers to the greater adoption of NoSQL stores include the use of low-level query languages, lack of standardized interfaces, and huge previous investments in existing relational databases.



NoSQL Database

- Most NoSQL stores **lack true ACID(Atomicity, Consistency, Isolation, Durability) transactions** but a few databases, such as **MarkLogic, Aerospike, FairCom c-treeACE, Google Spanner** (though technically a NewSQL database), **Symas LMDB, and OrientDB** have made them central to their designs.

////////////////////////////////////

NoSQL Database

- Most NoSQL databases **offer a concept of eventual consistency** in which **database changes** are propagated to **all nodes** so **queries for data** might not return updated data immediately or might result in reading data that is not **accurate** which is a problem known as stale reads.
- Also some NoSQL systems **may exhibit lost writes and other forms of data loss**. Some NoSQL systems provide concepts such as **write-ahead logging** to avoid data loss.



NoSQL Database

- One simple example of a NoSQL database is a document database. In a **document database**, **data is stored in documents** rather than tables.
- **Each document can contain a different set of fields, making it easy to accommodate changing data requirements**
- For example, “Take, for instance, a database that holds data regarding employees.”



NoSQL Database

- In a relational database, **this information might be stored in tables**, with one table for employee information and another table for department information.
- In a document database, each employee would be stored as a separate document, with all of their information contained within the document.



Key Features of NoSQL

- **Dynamic schema:** NoSQL databases do not have a fixed schema and can accommodate changing data structures without the need for migrations or schema alterations.
- **Horizontal scalability:** NoSQL databases are designed to scale out by adding more nodes to a database cluster, making them well-suited for handling large amounts of data and high levels of traffic.
- **Document-based:** Some NoSQL databases, such as MongoDB, use a document-based data model, where data is stored in semi-structured format, such as JSON or BSON.



Key Features of NoSQL

- **Key-value-based:** Other NoSQL databases, such as Redis, use a **key-value data model**, where data is stored as a collection of key-value pairs.
- **Column-based:** Some NoSQL databases, such as Cassandra, use a **column-based data model**, where data is organized into columns instead of rows.
- **Distributed and high availability:** NoSQL databases are often designed to be highly available and to automatically handle node failures and data replication across multiple nodes in a database cluster.



Key Features of NoSQL

- **Flexibility:** NoSQL databases allow developers to store and retrieve data in a flexible and dynamic manner, with support for multiple data types and changing data structures.
- **Performance:** NoSQL databases are optimized for high performance and can handle a high volume of reads and writes, making them suitable for big data and real-time applications.



Advantages of NoSQL

- There are many advantages of working with NoSQL databases such as MongoDB and Cassandra. The main advantages are high scalability and high availability.
 - **High scalability**
 - **Flexibility**
 - **High availability**
 - **Scalability**
 - **Performance**
 - **Cost effectiveness**
 - **Agility**



Advantages of NoSQL

- **High scalability** : NoSQL databases use **sharding** for horizontal scaling. **Partitioning of data and placing it on multiple machines** in such a way that the order of the data is preserved is sharding.
 - **Vertical scaling**
 - adding more resources to the existing machine
 - not that easy to implement
 - **Horizontal scaling** is
 - adding more machines to handle the data
 - easy to implement.
 - Examples of **horizontal scaling databases** are **MongoDB, Cassandra**, etc.
 - NoSQL can handle scalability, as the **data grows** NoSQL scale itself to handle that data in an efficient manner.



Advantages of NoSQL

- **Flexibility:**

- Designed to handle unstructured or semi-structured data,
- Can accommodate dynamic changes to the data model.
- Makes a good fit for applications that need to **handle changing data requirements**.

- **High availability :**

- Auto replication feature
- Makes it highly available, in case of any failure.



Advantages of NoSQL

- **Scalability:**

- Means that they **can handle large amounts of data and traffic with ease.**
- Makes them a good fit for applications that **need to handle large amounts of data or traffic**

- **Performance:**

- Means that they can **offer improved performance** compared to traditional relational databases.



Advantages of NoSQL

- **Cost-effectiveness:**

- More cost-effective than traditional relational databases
- Less complex and **do not require expensive hardware or software.**

- **Agility:**

- Ideal for agile development.



Disadvantages of NoSQL

- **Lack of standardization :**

- There are many different types of NoSQL databases, each with its own unique strengths and weaknesses.
- This lack of standardization can make it **difficult to choose the right database for a specific application**

- **Lack of ACID compliance :**

- NoSQL databases are **not fully ACID-compliant**, which means that **they do not guarantee the consistency, integrity, and durability of data.**
- This can be a **drawback for applications that require strong data consistency guarantees.**



Disadvantages of NoSQL

- **Narrow focus :**
 - Mainly **designed for storage** but it **provides very little functionality**.
 - Relational databases are a better choice in the field of Transaction Management than NoSQL.
- **Open-source :**
 - NoSQL is **open-source database**.
 - No reliable standard for NoSQL.
 - In other words, **two database systems are likely to be unequal**.
- **Lack of support for complex queries :**
 - NoSQL databases are **not designed to handle complex queries**
 - They are not a good fit for applications that require complex data analysis or reporting.



Disadvantages of NoSQL

- **Lack of maturity :**

- NoSQL databases are relatively **new and lack the maturity** of traditional relational databases.
- **Less reliable and less secure** than traditional databases.

- **Management challenge :**

- Data management in NoSQL is much **more complex** than in a relational database.
- NoSQL, in particular, has a reputation for **being challenging to install and even more hectic to manage on a daily basis.**



Disadvantages of NoSQL

- **GUI is not available :**
 - GUI mode tools to **access the database are not flexibly available** in the market.
- **Backup :**
 - Backup is a great weak point for some NoSQL databases like MongoDB.
- **Large document size :**
 - Some database systems like MongoDB and CouchDB store data in JSON format.
 - **Documents are quite large** (BigData, network bandwidth, speed), and having descriptive key names actually hurts since they increase the document size.



When should NoSQL be used

- **When a huge amount of data needs** to be stored and retrieved.
- The **relationship between the data you store is not that important**
- The **data changes over time** and is not structured.
- **Support of Constraints and Joins is not required** at the database level
- The **data is growing continuously and you need to scale the database regularly to handle the data.**



Classification of NoSQL Database

- NoSQL databases are **often used in applications where there is a high volume of data** that needs to be processed and analyzed in real-time, such as **social media analytics, e-commerce, and gaming**.
- They can also be used for other applications, such as **content management systems, document management, and customer relationship management**.



Classification of NoSQL Database

- NoSQL databases may **not be suitable** for all applications, as they **may not provide the same level of data consistency** and **transactional guarantees** as traditional relational databases.
- It is important to carefully evaluate the specific needs of an application when choosing a database management system.



Classification of NoSQL Database

- **Document databases:** These databases store data as semi-structured documents, such as JSON or XML, and can be *queried using document-oriented query languages*.
- **Key-value stores:** These databases store data as key-value pairs, and are optimized for simple and fast read/write operations.



Classification of NoSQL Database

- **Column-family stores:** These databases store data as column families, which are sets of columns that are treated as a single entity. They are **optimized for fast and efficient querying of large amounts of data.**
- **Graph databases:** These databases store data as nodes and edges, and are designed to handle complex relationships between data.



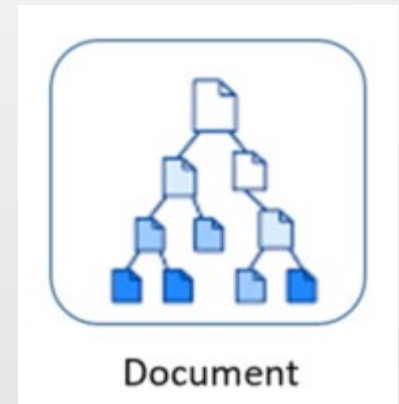
Types of NoSQL

- Types of NoSQL databases and the name of the databases system that falls in that category are:
 - **Graph Databases:** Examples – Amazon Neptune, Neo4j
 - **Key value store:** Examples – Memcached, Redis, Coherence
 - **Tabular:** Examples – Hbase, Big Table, Accumulo
 - **Document-based:** Examples – MongoDB, CouchDB, Cloudant



Document Database in NoSQL

Types of NoSQL





Document Database in NoSQL

- Documents
 - **Loosely structured sets of key/value pairs in documents**, e.g., XML, JSON, BSON
 - **Encapsulate and encode data in some standard formats** or encodings
 - Are **addressed in the database via a unique key**
 - **Documents are treated as a whole**, avoiding splitting a document into its constituent name/value pairs
- Allow documents retrieving by keys or contents
- Notable for:
 - MongoDB (used in FourSquare, Github, and more)
 - CouchDB (used in Apple, BBC, Canonical, Cern, and more)

Document Database in NoSQL

- The central concept is the **notion of a "document" which corresponds to a row in RDBMS.**
- A document comes in some standard **formats like JSON (BSON).**
- Documents are **addressed in the database via a unique key** that represents that document.
- The database offers an **API or query language that retrieves documents based on their contents.**
- **Documents are schema free**, i.e., different documents can have structures and schema that differ from one another. (An RDBMS requires that each row contain the same columns.)

Document Database in NoSQL

- A Document Data Model stores data in JSON, BSON, or XML documents.
- In this data model, we **can move documents under one document and apart from this**, any particular elements can be indexed to run queries faster.
- Often documents are stored and retrieved in such a way that it becomes close to the data objects which are used in many applications which **means very less translations are required to use data in applications**.
- JSON is **a native language that is often used to store and query data too**.

Document Database in NoSQL

- So in the document data model, each document has a key-value pair below is an example for the same.

```
{  
  "Name" : "Yashodhra",  
  "Address" : "Near Patel Nagar",  
  "Email" : "yahoo123@yahoo.com",  
  "Contact" : "12345"  
}
```



Document Database in NoSQL

- **Working of Document Data Model:**
- This is a data model which works **as a semi-structured data model** in which **the records and data associated with them are stored in a single document** which means this data model is not completely unstructured.
- The main thing is that data here is stored in a document.



Document Database in NoSQL

- **Features:**
- **Document Type Model:** As we all know data is stored in **documents rather than tables or graphs**, so it becomes **easy to map things** in many programming languages.
- **Flexible Schema:** Overall schema is very much flexible to support this statement one must know that **not all documents in a collection need to have the same fields**.



Document Database in NoSQL

- **Features:**
- **Distributed and Resilient:** Document data models are very much dispersed which is the **reason behind horizontal scaling** and distribution of data.
- **Manageable Query Language:** These data models are the ones in which query language **allows the developers to perform CRUD (Create Read Update Destroy) operations** on the data model.



Document Database in NoSQL

- **Examples of Document Data Models :**
 - **Amazon DocumentDB**
 - **MongoDB**
 - **Cosmos DB**
 - **ArangoDB**
 - **Couchbase Server**
 - **CouchDB**



Document Database in NoSQL

- **Advantages:**
- **Schema-less:** These are **very good in retaining existing data** at massive volumes because there are absolutely no restrictions in the format and the structure of data storage.
- **Faster creation of document and maintenance:** It is very **simple to create a document** and apart from this maintenance requires is almost nothing.



Document Database in NoSQL

- **Advantages:**
- **Open formats:** It has a very simple **build process that uses XML, JSON, and its other forms.**
- **Built-in versioning:** It has built-in versioning which means as the **documents grow in size there might be a chance they can grow in complexity.** Versioning decreases conflicts.

Document Database in NoSQL

- **Disadvantages:**
- **Weak Atomicity:** It lacks in supporting multi-document ACID transactions. **A change in the document data model involving two collections will require us to run two separate queries** i.e. one for each collection. This is where it breaks atomicity requirements.
- **Consistency Check Limitations:** One can search the collections and **documents that are not connected to an author collection but doing this might create a problem** in the performance of database performance.



Document Database in NoSQL

- **Disadvantages:**
- **Security:** Nowadays many web applications lack security which in turn results in the leakage of sensitive data. So it becomes a point of concern, one must pay attention to web app vulnerabilities.



Document Database in NoSQL

- **Applications of Document Data Model :**
- **Content Management:** These data models are very much used in creating various video streaming platforms, blogs, and similar services. Because each is stored as a single document and the database here is much easier to maintain as the service evolves over time.
- **Book Database:** These are very much useful in making book databases because as we know this data model lets us nest.



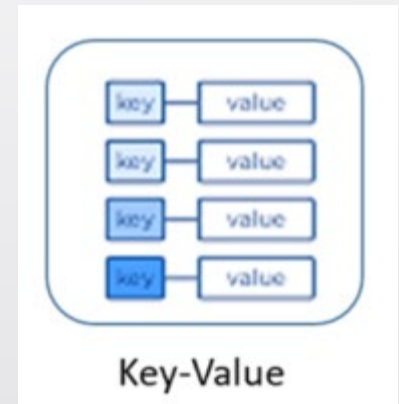
Document Database in NoSQL

- **Applications of Document Data Model :**
- **Catalog:** When it comes to storing and reading catalog files these data models are very much used because it has a fast reading ability if incase Catalogs have thousands of attributes stored.
- **Analytics Platform:** These data models are very much used in the Analytics Platform.



Key-Value Data Model in NoSQL

Types of NoSQL



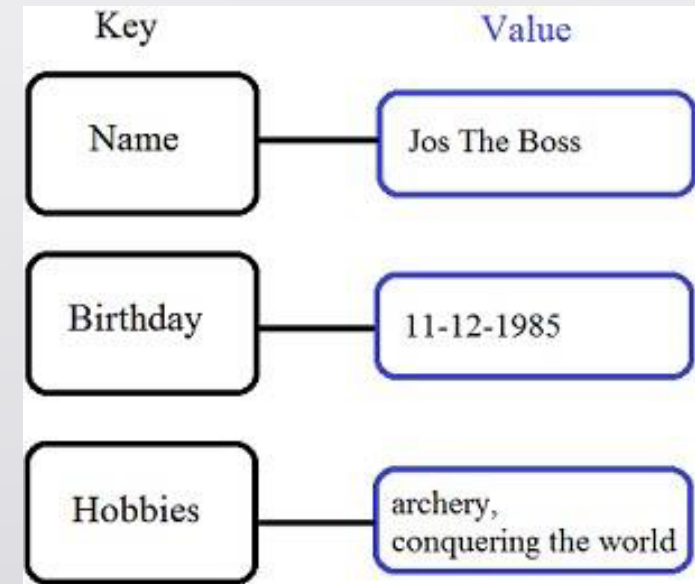


Key-Value Data Model in NoSQL

- A key-value data model or database is also referred to as a **key-value store**.
- It is a non-relational type of database. In this, an associative array is **used as a basic database in which an individual key is linked with just one value in a collection**.
- For the values, **keys are special identifiers**.
- **Any kind of entity can be valued**.
- The collection of key-value pairs stored on separate records is called **key-value databases** and they do not have an already defined structure.

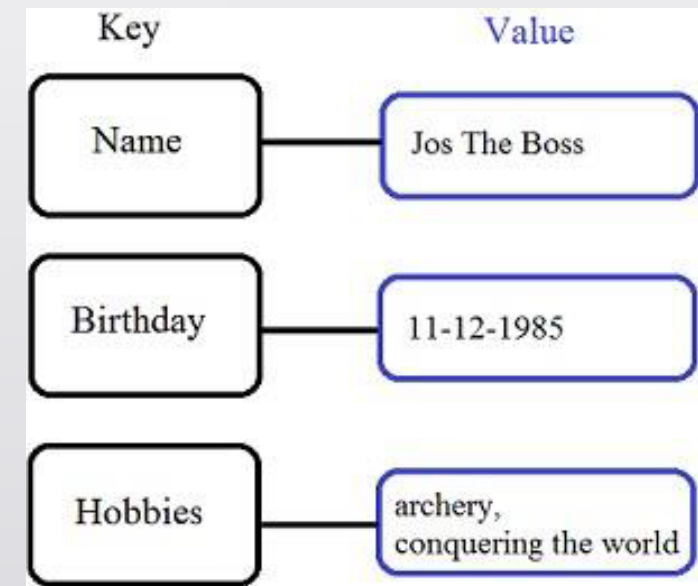
Key-Value Data Model in NoSQL

- **Store data in a schema-less way**
- Store data as maps
 - HashMaps or associative arrays
 - Provide a very efficient average running time algorithm for accessing data
- Notable for:
 - Couchbase (Zynga, Vimeo, NAVTEQ, ...)
 - Redis (Craiglist, Instagram, StackOverflow, flickr, ...)
 - Amazon Dynamo (Amazon, Elsevier, IMDb, ...)
 - Apache Cassandra (Facebook, Digg, Reddit, Twitter,...)
 - Voldemort (LinkedIn, eBay, ...)
 - Riak (Github, Comcast, Mochi, ...)



Key-Value Data Model in NoSQL

- **Store data in a schema-less way**
- Store data as maps
 - HashMaps or associative arrays
 - Provide a very efficient average running time algorithm for accessing data
- Notable for:
 - Couchbase (Zynga, Vimeo, NAVTEQ, ...)
 - Redis (Craiglist, Instagram, StackOverflow, flickr, ...)
 - Amazon Dynamo (Amazon, Elsevier, IMDb, ...)
 - Apache Cassandra (Facebook, Digg, Reddit, Twitter,...)
 - Voldemort (LinkedIn, eBay, ...)
 - Riak (Github, Comcast, Mochi, ...)



How do Key-Value Data Model work?

- A **number of easy strings or even a complicated entity are referred to as a value** that is associated with a key by a key-value database, which is utilized to monitor the entity.
- Like in many programming paradigms, a key-value database **resembles a map object or array, or dictionary**, however, which is put away in a tenacious manner and controlled by a DBMS.

How do Key-Value Database work?

- An efficient and compact structure of the **index is used** by the key-value store to have the option to rapidly and dependably find value using its key.
- For example, Redis is a key-value store **used to tracklists, maps, heaps, and primitive types** (which are simple data structures) in a constant database.
- **Redis** can uncover a very basic point of interaction to query and manipulate value types, just by supporting a predetermined number of value types, and when arranged, is prepared to do high throughput.



When to use a Key-Value Database

- Here are a few situations in which you can use a key-value database:-
 - **User session attributes in an online app** like finance or gaming, which is referred to as real-time random data access.
 - **Caching mechanism for repeatedly accessing data or key-based design.**
 - The **application is developed on queries that are based on keys.**



Key-Value Database

- **Features:**
- One of the most un-complex kinds of NoSQL data models.
- For **storing, getting, and removing** data, key-value databases **utilize simple functions.**
- **Querying language is not present in key-value databases.**
- **Built-in redundancy makes this database more reliable.**



Key-Value Database

- **Advantages:**
- It is **very easy to use**. Due to the simplicity of the database, data can accept any kind, or even different kinds when required.
- Its **response time is fast** due to its simplicity, given that the remaining environment near it is very much constructed and improved.
- Key-value store databases are **scalable vertically as well as horizontally**.
- Built-in redundancy makes this database **more reliable**.



Key-Value Database

- **Disadvantages:**
- As **querying language is not present** in key-value databases, transportation of queries from one database to a different database cannot be done.
- The **key-value store database is not refined**. You cannot query the database without a key.



Key-Value Database

- **Popular key-value databases which are widely used:**
- **Couchbase:** It permits SQL-style querying and searching for text.
- **Amazon DynamoDB:** The key-value database which is mostly used is Amazon DynamoDB as it is a trusted database used by a large number of users. It can easily handle a large number of requests every day and it also provides various security options.
- **Riak:** It is the database used to develop applications.
- **Aerospike:** It is an open-source and real-time database working with billions of exchanges.
- **Berkeley DB:** It is a high-performance and open-source database providing scalability.



Columnar Data Model in NoSQL

Types of NoSQL



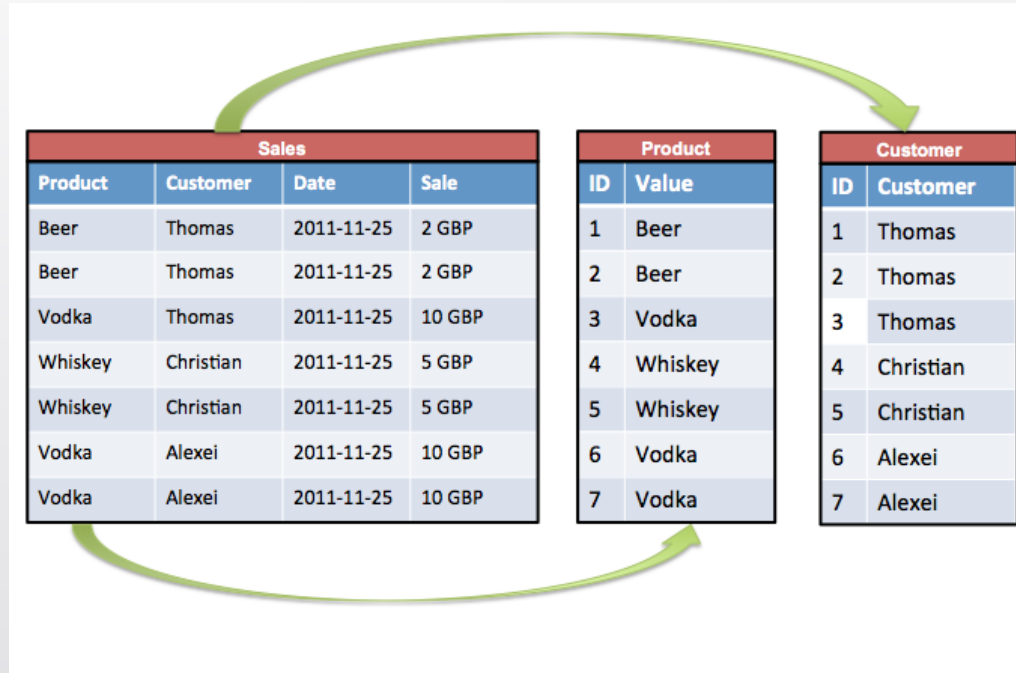


Columnar Data Model of NoSQL

- The Columnar Data Model of NoSQL is important. NoSQL databases are different from SQL databases.
- This is because it uses a data model that has a different structure than the previously followed row-and-column table model used with relational database management systems (RDBMS).
- NoSQL databases are a flexible schema model which is designed to scale horizontally across many servers and is used in large volumes of data.

Columnar Data Model of NoSQL

- Data are **stored in a column-oriented way**
 - Data efficiently stored
 - Avoids consuming space for storing nulls
 - Columns are grouped in column-families
 - Data is stored by column families
 - Unit of data is a set of key/value pairs
 - Identified by “row-key”
 - Ordered and sorted based on row-key
- Notable for:
 - Google's Bigtable (used in all Google's services)
 - HBase (Facebook, StumbleUpon, Hulu, Yahoo!, ...)





Columnar Data Model of NoSQL

- Relational database stores data in rows and also reads the data row by row, column store is organized as a set of columns.
- So if someone **wants to run analytics on a small number of columns**, one can read those columns directly **without consuming memory with the unwanted data**.
- **Columns are somehow are of the same type and gain from more efficient compression, which makes reads faster than before.**
- Examples of Columnar Data Model: **Cassandra and Apache Hadoop Hbase.**

Working of Columnar Data Model

- In Columnar Data Model instead of **organizing information into rows, it does in columns.**
- This makes them function the same way that tables work in relational databases.
- This type of data model is much **more flexible obviously because it is a type of NoSQL database.**
- The below example will help in understanding the Columnar data model: **Row-Oriented Table, Column-Oriented Table.**

Working of Columnar Data Model

- **Row-Oriented Table**

S.No.	Name	Course	Branch	ID
01.	Tanmay	B-Tech	Computer	2
02.	Abhishek	B-Tech	Electronics	5
03.	Samriddha	B-Tech	IT	7
04.	Aditi	B-Tech	E & TC	8

Working of Columnar Data Model

- **Column-Oriented Table** - uses the concept of keyspace, which is like a schema in relational models.

S.No.	Name	ID
01.	Tanmay	2
02.	Abhishek	5
03.	Samriddha	7
04.	Aditi	8

S.No.	Course	ID
01.	B-Tech	2
02.	B-Tech	5
03.	B-Tech	7
04.	B-Tech	8

S.No.	Branch	ID
01.	Computer	2
02.	Electronics	5
03.	IT	7
04.	E & TC	8



Advantage of Columnar Data Model

- **Well structured**: Since these data models are good at compression so these are very structured or well organized in terms of storage.
- **Flexibility**: A large amount of flexibility as it is not necessary for the columns to look like each other, which means one can add new and different columns without disrupting the whole database



Advantage of Columnar Data Model

- **Aggregation queries are fast:** The most important thing is aggregation queries are quite fast because a majority of the information is stored in a column. An example would be Adding up the total number of students enrolled in one year.
- **Scalability:** It can be spread across large clusters of machines, even numbering in thousands.
- **Load Times:** Since one can easily load a row table in a few seconds so load times are nearly excellent.

Disadvantage of Columnar Data Model

- **Designing indexing Schema:** To design an effective and working schema is too difficult and very time-consuming.
- **Suboptimal data loading:** incremental data loading is suboptimal and must be avoided, but this might not be an issue for some users.
- **Security vulnerabilities:** If security is one of the priorities then it must be known that the Columnar data model lacks inbuilt security features in this case, one must look into relational databases.
- **Online Transaction Processing (OLTP):** Online Transaction Processing (OLTP) applications are also not compatible with columnar data models because of the way data is stored.



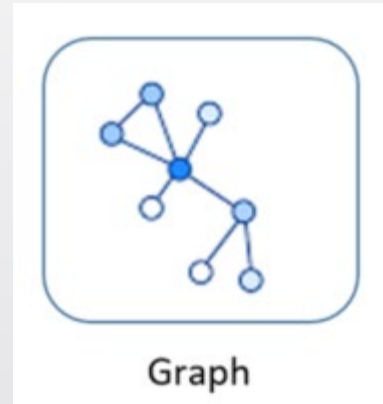
Applications of Columnar Data Model

- Columnar Data Model is very much used in various **Blogging Platforms.**
- It is used in **Content management systems like WordPress, Joomla, etc.**
- It is used in **Systems that maintain counters.**
- It is used in **Systems that require heavy write requests.**
- It is used in **Services that have expiring usage.**



Graph Data Model in NoSQL

Types of NoSQL



Graph Based Data Model in NoSQL

- Graph Based Data Model in NoSQL is a type of Data Model which tries to **focus on building the relationship between data elements**.
- As the name suggests Graph-Based Data Model, **each element here is stored as a node, and the association between these elements is often known as Links**.
- **Association is stored directly as these are the first-class elements of the data model**. These data models give us a conceptual view of the data.

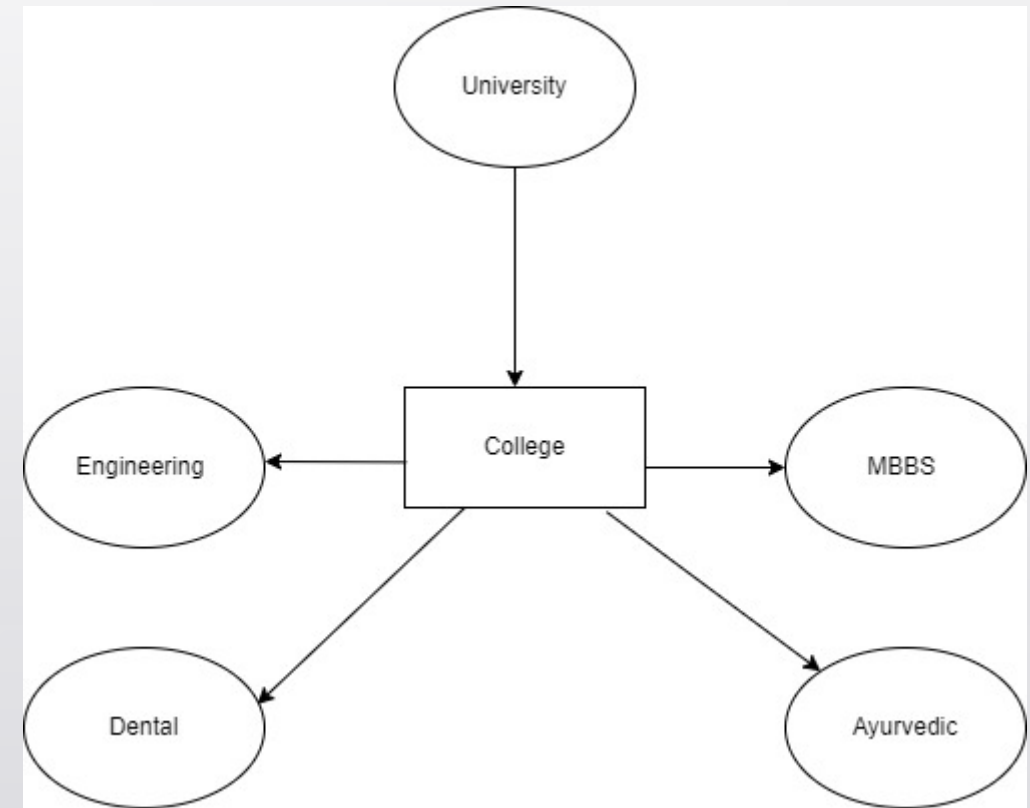


Graph Based Data Model in NoSQL

- These are the data models which are **based on topographical network structure**. Obviously, in graph theory, we have terms like Nodes, edges, and properties, let's see what it means here in the Graph-Based data model.
- **Nodes**: These are the instances of data that represent objects which is to be tracked.
- **Edges**: As we already know edges represent relationships between nodes.
- **Properties**: It represents information associated with nodes.

Graph Based Data Model in NoSQL

- The below image represents Nodes with properties from relationships represented by edges.





Working of Graph Based Data Model

- In these data models, the **nodes** which are connected together **are connected physically and the physical connection among them is also taken as a piece of data.**
- Connecting data in this way becomes **easy to query a relationship.**
- This data model **reads the relationship from storage directly** instead of calculating and querying the connection steps.
- Like many different NoSQL databases these **data models don't have any schema** as it is important because schema makes the model well and good and easy to edit.



Examples of Graph Based Data Model

- **JanusGraph**: These are very helpful in big data analytics. It is a scalable graph database system open source too. JanusGraph has different features like:
 - **Storage**: Many options are available for storing graph data like Cassandra.
 - **Support for transactions**: There are many supports available like ACID (Atomicity, Consistency, Isolation, and Durability) which can hold thousands of concurrent users.
 - **Searching options**: Complex searching options are available and optional support too.



Examples of Graph Based Data Model

- **Neo4j**: It stands for Network Exploration and Optimization 4 Java. As the name suggests this graph database is written in Java with native graph storage and processing. Neo4j has different features like:
 - **Scalable**: Scalable through data partitioning into pieces known as shards.
 - **Higher Availability**: Availability is very much high due to continuous backups and rolling upgrades.
 - **Query Language**: Uses programmer-friendly query language Cypher graph query language. **DGraph** main features are:



Examples of Graph Based Data Model

- **DGraph**: It is an open-source distributed graph database system designed with scalability.
 - **Query Language**: It uses **GraphQL**, which is solely made for APIs.
 - **open-source system**: support for many open standards.



Advantages of Graph Based Data Model

- **Structure**: The structures are very agile and workable too.
- **Explicit Representation**: The portrayal of relationships between entities is explicit.
- **Real-time O/P Results**: Query gives us real-time output results.



Disadvantages of Graph Based Data Model

- **No standard query language**: Since the language depends on the platform that is used so there is **no certain standard query language**.
- **Unprofessional Graphs**: Graphs are **very unprofessional** for transactional-based systems.
- **Small User Base**: The **user base** is small which makes it very difficult to get support when running into a system.



Applications of Graph Based Data Model

- Graph data models are very much **used in fraud detection** which itself is very much useful and important.
- It is used in **Digital asset management** which provides a scalable database model to keep track of digital assets.
- It is used in **Network management** which alerts a network administrator about problems in a network.
- It is used in **Context-aware services** by giving traffic updates and many more.
- It is used in **Real-Time Recommendation Engines** which provide a better user experience.



NoSQL

- NoSQL databases offer several benefits over traditional relational databases, such as scalability, flexibility, and cost-effectiveness.
- However, they also have several drawbacks, such as a lack of standardization, lack of ACID compliance, and lack of support for complex queries.
- When choosing a database for a specific application, it is important to weigh the benefits and drawbacks carefully to determine the best fit.



NoSQL dealing with Big Data and Scalability

- NoSQL solutions are designed to run on clusters or multi-node database solutions
- Larger volumes of data/operations requires to add more machines to the cluster, Known as 'scaling out' or 'horizontal scaling'
- Different approaches include:
 - **Master-slave**
 - **Sharding (partitioning)**



Scaling in NoSQL

- **Master-Slave**

- All writes are written to the master. All reads performed against the replicated slave databases
- Critical reads may be incorrect as writes may not have been propagated down
- Large data sets can pose problems as master needs to duplicate data to

- **Sharding**

- Any DB distributed across multiple machines needs to know in what machine a piece of data is stored or must be stored
- A sharding system makes this decision for each row, using its key



No ACID - NoSQL

- NoSQL
 - Does not give importance to ACID properties
 - In some cases completely ignores them
- In distributed parallel systems it is difficult/impossible to ensure ACID properties
 - Long-running transactions don't work because keeping resources blocked for a long time is not practical



BASE Transactions - NoSQL

- Acronym contrived to be the opposite of ACID
 - **B**asically **A**vailable,
 - **S**oft state,
 - **E**ventually Consistent
- **Characteristics**
 - Weak consistency – stale data OK
 - Availability first
 - Best effort
 - Approximate answers OK
 - Aggressive (optimistic)
 - Simpler and faster



CAPE Theorem - NoSQL

- A congruent and logical way for assessing the problems involved in assuring ACID-like guarantees in distributed systems is provided by the CAP theorem
- At most two of the following three can be maximized at one time
 - **C**onsistency
 - Each client has the same view of the data
 - **A**vailability
 - Each client can always read and write
 - **P**artition tolerance
 - System works well across distributed physical networks

Visual Guide to NoSQL Systems



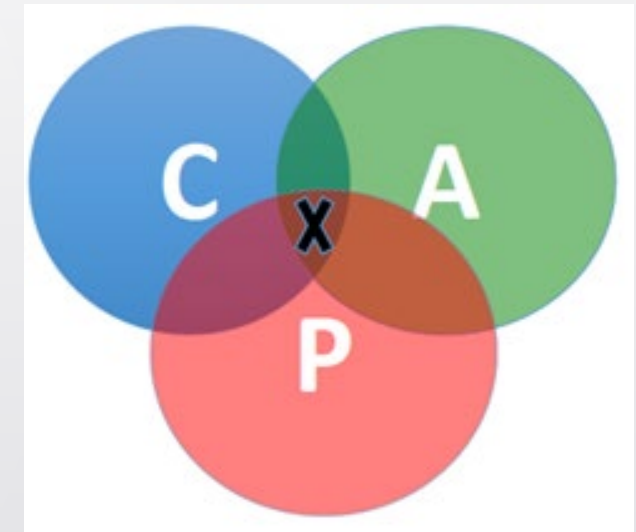


CAPE Theorem - NoSQL

- CAP theorem – At most two properties on three can be addressed
- The choices could be as follows:
 - Availability is compromised but consistency and partition tolerance are preferred over it
 - The system has little or no partition tolerance. Consistency and availability are preferred
 - Consistency is compromised but systems are always available and can work when parts of it are partitioned

CAPE Theorem - NoSQL

- Consistency and Availability is not “binary” decision
- AP systems relax consistency in favor of availability – but are not inconsistent
- CP systems sacrifice availability for consistency- but are not unavailable
- This suggests both AP and CP systems can offer a degree of consistency, and availability, as well as partition tolerance





Performance- NoSQL

- There is no perfect NoSQL database
- Every database has its advantages and disadvantages
 - Depending on the type of tasks (and preferences) to accomplish
- NoSQL is a set of concepts, ideas, technologies, and software dealing with
 - Big data
 - Sparse un/semi-structured data
 - High horizontal scalability
 - Massive parallel processing
- Different applications, goals, targets, approaches need different NoSQL solutions



Application area - NoSQL

- NoSQL preferred in domain where a large set of uncontrolled, unstructured, data that you are trying to fit into a RDBMS?
 - Log Analysis
 - Social Networking Feeds (many firms hooked in through Facebook or Twitter)
 - External feeds from partners
 - Data that is not easily analyzed in a RDBMS such as time-based data
 - Large data feeds that need to be massaged before entry into an RDBMS



DBA - NoSQL

- DBA and SysAdmin need to address:
 - Backups & recovery
 - Capacity planning
 - Performance monitoring
 - Data integration
 - Tuning & optimization



Module 6

BCSE302L – Database Systems